

```

1  /*
2  * File:    Mc32gestSpiFT800.h
3  * Author:  roestilo
4  *
5  * Created on 18. août 2014, 13:32
6  */
7
8  #ifndef MC32GESTSPIFT800_H
9  #define MC32GESTSPIFT800_H
10
11 #include "Mc32SpiUtil.h"
12 #include "FT_GPU.h"
13
14 #ifdef __cplusplus
15 extern "C" {
16 #endif
17     // Definition des commandes d'hôte
18 #define ACTIVE    0x00
19 #define STANDBY   0x41
20     // #define SLEEP    0x42
21 #define PWRDOWN   0x50
22 #define CLKEXT    0x44
23 #define CLK48M    0x62
24 #define CLK36M    0x61
25 #define CORERST   0x68
26
27     void HostCommand(uint8_t command);
28     void HostCommandDummyRead(void);
29     void InitFT800(void);
30     void wr8(uint32_t Addr, uint8_t Value);
31     void wr16(uint32_t Addr, uint16_t Value);
32     void wr32(uint32_t Addr, uint32_t Value);
33     uint8_t wr8s(uint32_t Addr, const char *S);
34     uint8_t rd8(uint32_t Addr);
35     uint16_t rd16(uint32_t Addr);
36     uint32_t rd32(uint32_t Addr);
37     void cmd(uint32_t command);
38     void dl(uint32_t instruction);
39     void cmd_incrementn(unsigned char n);
40     void dl_incrementn(unsigned char n);
41
42
43
44 #define VERTEX2F(x,y) (((1UL<<30)|(((x)&32767UL)<<15)|(((y)&32767UL)
45     <<0)))
46 #define VERTEX2II(x,y,handle,cell) (((2UL<<30)|(((x)&511UL)<<21)|(((y)
47     &511UL)<<12)|(((handle)&31UL)<<7)|(((cell)&127UL)<<0)))
48 #define BITMAP_SOURCE(addr) (((1UL<<24)|(((addr)&1048575UL)<<0))) // Non
49     documenté
50 #define CLEAR_COLOR_RGB(red,green,blue) (((2UL<<24)|(((red)&255UL)<<16)|
51     (((green)&255UL)<<8)|(((blue)&255UL)<<0)))
52 #define TAG(s) (((3UL<<24)|(((s)&255UL)<<0)))
53 #define COLOR_RGB(red,green,blue) (((4UL<<24)|(((red)&255UL)<<16)|

```

```

    (((green)&255UL)<<8)|(((blue)&255UL)<<0))
50 #define BITMAP_HANDLE(handle) ((5UL<<24)|(((handle)&31UL)<<0)) // Non documenté
51 #define CELL(cell) ((6UL<<24)|(((cell)&127UL)<<0)) // Non documenté
52 // Non documenté
53 #define BITMAP_LAYOUT(format,linstride,height) ((7UL<<24)|(((format)
    &31UL)<<19)|(((linstride)&1023UL)<<9)|(((height)&511UL)<<0))
54 #define BITMAP_LAYOUT_H(linstride, height) ((40UL<<24) |
    (((linstride)&0x03)<<2) | ((height)&0x03))
55
56 // Non documenté
57 #define BITMAP_SIZE(filter,wrapx,wrapy,width,height) ((8UL<<24)|
    (((filter)&1UL)<<20)|(((wrapx)&1UL)<<19)|(((wrapy)&1UL)<<18)|
    (((width)&511UL)<<9)|(((height)&511UL)<<0))
58 #define BITMAP_SIZE_H(width, height) ((41UL<<24) | (((width)&0x03)<<2)
    | ((height)&0x03))
59
60 // Non documenté
61 #define ALPHA_FUNC(func,ref) ((9UL<<24)|(((func)&7UL)<<8)|(((ref)
    &255UL)<<0))
62 #define STENCIL_FUNC(func,ref,mask) ((10UL<<24)|(((func)&7UL)<<16)|
    (((ref)&255UL)<<8)|(((mask)&255UL)<<0))
63 // Non documenté
64 #define BLEND_FUNC(src,dst) ((11UL<<24)|(((src)&7UL)<<3)|(((dst)&7UL)
    <<0))
65 #define STENCIL_OP(sfail,spass) ((12UL<<24)|(((sfail)&7UL)<<3)|
    (((spass)&7UL)<<0))
66 #define POINT_SIZE(size) ((13UL<<24)|(((size)&8191UL)<<0))
67 #define LINE_WIDTH(width) ((14UL<<24)|(((width)&4095UL)<<0))
68 // Non documenté
69 #define CLEAR_COLOR_A(alpha) ((15UL<<24)|(((alpha)&255UL)<<0))
70 // Non documenté
71 #define COLOR_A(alpha) ((16UL<<24)|(((alpha)&255UL)<<0))
72 // Non documenté
73 #define CLEAR_STENCIL(s) ((17UL<<24)|(((s)&255UL)<<0))
74 // Non documenté
75 #define CLEAR_TAG(s) ((18UL<<24)|(((s)&255UL)<<0))
76 // Non documenté
77 #define STENCIL_MASK(mask) ((19UL<<24)|(((mask)&255UL)<<0))
78 // Non documenté
79 #define TAG_MASK(mask) ((20UL<<24)|(((mask)&1UL)<<0))
80 #define BITMAP_TRANSFORM_A(a) ((21UL<<24)|(((a)&131071UL)<<0))// Non documenté
81 #define BITMAP_TRANSFORM_B(b) ((22UL<<24)|(((b)&131071UL)<<0))// Non documenté
82 #define BITMAP_TRANSFORM_C(c) ((23UL<<24)|(((c)&16777215UL)<<0))// Non documenté
83 #define BITMAP_TRANSFORM_D(d) ((24UL<<24)|(((d)&131071UL)<<0))// Non documenté
84 #define BITMAP_TRANSFORM_E(e) ((25UL<<24)|(((e)&131071UL)<<0))// Non documenté
85 #define BITMAP_TRANSFORM_F(f) ((26UL<<24)|(((f)&16777215UL)<<0))// Non documenté

```

```

documenté
86 #define SCISSOR_XY(x,y) ((27UL<<24)|(((x)&511UL)<<9)|(((y)&511UL)<<0))
87 #define SCISSOR_SIZE(width,height) ((28UL<<24)|(((width)&1023UL)<<10)|  ↗
    (((height)&1023UL)<<0))
88 // Non documenté
89 #define CALL(dest) ((29UL<<24)|(((dest)&65535UL)<<0))
90 // Non documenté
91 #define JUMP(dest) ((30UL<<24)|(((dest)&65535UL)<<0))
92 #define BEGIN(prim) ((31UL<<24)|(((prim)&15UL)<<0))
93 // Non documenté
94 #define COLOR_MASK(r,g,b,a) ((32UL<<24)|(((r)&1UL)<<3)|(((g)&1UL)<<2)|  ↗
    (((b)&1UL)<<1)|(((a)&1UL)<<0))
95 #define CLEAR(c,s,t) ((38UL<<24)|(((c)&1UL)<<2)|(((s)&1UL)<<1)|(((t)  ↗
    &1UL)<<0))
96 #define END() ((33UL<<24))
97 // Non documenté
98 #define SAVE_CONTEXT() ((34UL<<24))
99 // Non documenté
100 #define RESTORE_CONTEXT() ((35UL<<24))
101 // Non documenté
102 #define RETURN() ((36UL<<24))
103 // Non documenté
104 #define MACRO(m) ((37UL<<24)|(((m)&1UL)<<0))
105 #define DISPLAY() ((0UL<<24))
106
107 #define FT_GPU_NUMCHAR_PERFONT (128)
108 #define FT_GPU_FONT_TABLE_SIZE (148)
109
110 /* FT800 font table structure */
111 /* Font table address in ROM can be found by reading the address  ↗
    from 0xFFFFC location. */
112
113 /* 16 font tables are present at the address read from location  ↗
    0xFFFFC */
114 typedef struct FT_Gpu_Fonts {
115     /* All the values are in bytes */
116     /* Width of each character font from 0 to 127 */
117     uint8_t FontWidth[FT_GPU_NUMCHAR_PERFONT];
118     /* Bitmap format of font wrt bitmap formats supported by FT800  ↗
        - L1, L4, L8 */
119     uint32_t FontBitmapFormat;
120     /* Font line stride in FT800 ROM */
121     uint32_t FontLineStride;
122     /* Font width in pixels */
123     uint32_t FontWidthInPixels;
124     /* Font height in pixels */
125     uint32_t FontHeightInPixels;
126     /* Pointer to font graphics raw data */
127     uint32_t PointerToFontGraphicsData;
128 } FT_Gpu_Fonts_t;
129
130 #define cmd_exec() wr16(REG_CMD_WRITE, cmd_offset)
131 extern unsigned short cmd_offset;

```

```

...nstrateur_graphique\firmware\src\Mc32gestSpiFT800.h 4
132 #define cmd_wait() cmd_offset=rd16(REG_CMD_WRITE);while(rd16  ↗
    (REG_CMD_READ) != cmd_offset)
133
134 // Pour correspondre aux fonctions données dans le programmeurs  ↗
    guide
135 int _dummy;
136 #define tr8(value) _dummy = spi_read1(value);
137 // #define tr16(value) tr8((value) & 0xFF);tr8(((value) >> 8) &  ↗
    0xFF) //LSB first
138 #define tr16(value) tr8(((value) >> 8) & 0xFF); tr8((value) & 0xFF)
139 // #define tr32(value) tr16((value) & 0xFFFF);tr16(((value) >> 16) &  ↗
    0xFFFF) //LSB first
140 #define tr32(value) tr8(((value) >> 24) & 0xFF); tr8(((value) >> 16) &  ↗
    0xFF); tr8(((value) >> 8) & 0xFF); tr8((value) & 0xFF)
141
142
143 #define cmd_increment4() cmd_incrementn(n:4)
144 #define cmd_increment2() cmd_incrementn(n:2)
145 /**
146  * start a new display list
147  */
148 #define cmd_dlstart() dl(CMD_DLSTART)
149
150 /**
151  * swap the current display list
152  */
153 #define cmd_swap() cmd(CMD_DLSWAP)
154
155 /**
156  * draw text
157  */
158 #define cmd_text(x, y, font, options, s) cmd(CMD_TEXT);wr16(RAM_CMD  ↗
    +cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    y);cmd_increment2();wr16(RAM_CMD+cmd_offset, font);cmd_increment2  ↗
    ();wr16(RAM_CMD+cmd_offset, options);cmd_increment2();cmd_incrementn  ↗
    (n:wr8s(RAM_CMD+cmd_offset, s))
159
160 /**
161  * draw a decimal number
162  */
163 #define cmd_number(x, y, font, options, n) cmd(CMD_NUMBER);wr16(RAM_CMD  ↗
    +cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    y);cmd_increment2();wr16(RAM_CMD+cmd_offset, font);cmd_increment2  ↗
    ();wr16(RAM_CMD+cmd_offset, options);cmd_increment2();cmd(n)
164
165 /**
166  * draw a button
167  */
168 #define cmd_button(x, y, w, h, font, options, s) cmd(CMD_BUTTON);wr16  ↗
    (RAM_CMD+cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    y);cmd_increment2();wr16(RAM_CMD+cmd_offset, w);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, h);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    font);cmd_increment2();wr16(RAM_CMD+cmd_offset,

```

```
options);cmd_increment2();cmd_incrementn(n:wr8s(RAM_CMD+cmd_offset,
s))
169
170 /**
171  * draw an analog clock
172  */
173 #define cmd_clock(x, y, r, options, h, m, s, ms) cmd(CMD_CLOCK);wr16
(RAM_CMD+cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,
y);cmd_increment2();wr16(RAM_CMD+cmd_offset, r);cmd_increment2();wr16
(RAM_CMD+cmd_offset, options);cmd_increment2();wr16(RAM_CMD
+cmd_offset, h);cmd_increment2();wr16(RAM_CMD+cmd_offset,
m);cmd_increment2();wr16(RAM_CMD+cmd_offset, s);cmd_increment2();wr16
(RAM_CMD+cmd_offset, ms);cmd_increment2()
174
175 /**
176  * set the foreground color
177  */
178 #define cmd_fgcolor(c) cmd(CMD_FG_COLOR);cmd(c)
179
180 /**
181  * set the background color
182  */
183 #define cmd_bgcolor(c) cmd(CMD_BG_COLOR);cmd(c)
184
185 /**
186  * set the 3D effects for CMD_BUTTON and CMD_KEYS highlight color
187  */
188 #define cmd_gradcolor(c) cmd(CMD_GRAD_COLOR);cmd(c)
189
190 /**
191  * draw a gauge
192  */
193 #define cmd_gauge(x, y, r, options, major, minor, val, range) cmd
(CMD_GAUGE);wr16(RAM_CMD+cmd_offset, x);cmd_increment2();wr16(RAM_CMD
+cmd_offset, y);cmd_increment2();wr16(RAM_CMD+cmd_offset,
r);cmd_increment2();wr16(RAM_CMD+cmd_offset, options);cmd_increment2
();wr16(RAM_CMD+cmd_offset, major);cmd_increment2();wr16(RAM_CMD
+cmd_offset, minor);cmd_increment2();wr16(RAM_CMD+cmd_offset,
val);cmd_increment2();wr16(RAM_CMD+cmd_offset, range);cmd_increment2
()
194
195 /**
196  * draw a draw a smooth color gradient
197  */
198 #define cmd_gradient(x0, y0, rgb0, x1, y1, rgb1) cmd(CMD_GRADIENT);wr16
(RAM_CMD+cmd_offset, x0);cmd_increment2();wr16(RAM_CMD+cmd_offset,
y0);cmd_increment2();cmd(rgb0);wr16(RAM_CMD+cmd_offset,
x1);cmd_increment2();wr16(RAM_CMD+cmd_offset, y1);cmd_increment2
();cmd(rgb1)
199
200 /**
201  * draw a row of keys
202  */
```

```

...nstrateur_graphique\firmware\src\Mc32gestSpiFT800.h 6
203 #define cmd_keys(x, y, w, h, font, options, s) cmd(CMD_KEYS);wr16  ↗
    (RAM_CMD+cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    y);cmd_increment2();wr16(RAM_CMD+cmd_offset, w);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, h);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    font);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    options);cmd_increment2();cmd_incrementn(n:wr8s(RAM_CMD+cmd_offset,  ↗
    s))
204
205 /**
206  * draw a progress bar
207  */
208 #define cmd_progress(x, y, w, h, options, val, range) cmd  ↗
    (CMD_PROGRESS);wr16(RAM_CMD+cmd_offset, x);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, y);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    w);cmd_increment2();wr16(RAM_CMD+cmd_offset, h);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, options);cmd_increment2();wr16(RAM_CMD  ↗
    +cmd_offset, val);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    range);cmd_increment2()
209
210 /**
211  * draw a draw a scroll bar
212  */
213 #define cmd_scrollbar(x, y, w, h, options, val, size, range) cmd  ↗
    (CMD_SCROLLBAR);wr16(RAM_CMD+cmd_offset, x);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, y);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    w);cmd_increment2();wr16(RAM_CMD+cmd_offset, h);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, options);cmd_increment2();wr16(RAM_CMD  ↗
    +cmd_offset, val);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    size);cmd_increment2();wr16(RAM_CMD+cmd_offset, range);cmd_increment2  ↗
    ()
214
215 /**
216  * draw a slider
217  */
218 #define cmd_slider(x, y, w, h, options, val, range) cmd  ↗
    (CMD_SLIDER);wr16(RAM_CMD+cmd_offset, x);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, y);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    w);cmd_increment2();wr16(RAM_CMD+cmd_offset, h);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, options);cmd_increment2();wr16(RAM_CMD  ↗
    +cmd_offset, val);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    range);cmd_increment2()
219
220 /**
221  * draw a rotary dial control
222  */
223 #define cmd_dial(x, y, r, options, val) cmd(CMD_DIAL);wr16(RAM_CMD  ↗
    +cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,  ↗
    y);cmd_increment2();wr16(RAM_CMD+cmd_offset, r);cmd_increment2();wr16  ↗
    (RAM_CMD+cmd_offset, options);cmd_increment2();wr16(RAM_CMD  ↗
    +cmd_offset, val)
224
225 /**
226  * draw a toggle switch

```

```

227     */
228 #define cmd_toggle(x, y, w, font, options, state, s) cmd      ↗
    (CMD_TOGGLE);wr16(RAM_CMD+cmd_offset, x);cmd_increment2();wr16      ↗
    (RAM_CMD+cmd_offset, y);cmd_increment2();wr16(RAM_CMD+cmd_offset,      ↗
    w);cmd_increment2();wr16(RAM_CMD+cmd_offset, font);cmd_increment2      ↗
    ();wr16(RAM_CMD+cmd_offset, options);cmd_increment2();wr16(RAM_CMD      ↗
    +cmd_offset, state);cmd_incrementn(n:wr8s(RAM_CMD+cmd_offset, s))
229
230 /**
231  * compute a CRC-32 for memory
232  */
233 #define cmd_memcrc(ptr, num, result) cmd(CMD_MEMCRC);cmd(ptr);cmd      ↗
    (num);unsigned short cmd_memcrc_result_ptr=cmd_offset;cmd      ↗
    (result);cmd_wait();result=rd32(cmd_memcrc_result_ptr)
234
235 /**
236  * write zero to a block of memory
237  */
238 #define cmd_memzero(ptr, num) cmd(CMD_MEMCRC);cmd(ptr);cmd(num)
239
240 /**
241  * fill memory with a byte value
242  */
243 #define cmd_memset(ptr, value, num) cmd(CMD_MEMCRC);cmd(ptr);cmd      ↗
    (value);cmd(num)
244
245 /**
246  * write bytes into memory
247  */
248 #define cmd_memwrite(ptr, num)      cmd(CMD_MEMWRITE);cmd(ptr);cmd(num)
249
250 /**
251  * copy a block of memory
252  */
253 #define cmd_memcpy(dest, src, num) cmd(CMD_MEMCRC);cmd(dest);cmd      ↗
    (src);cmd(num)
254
255 /**
256  * append memory to display list
257  */
258 #define cmd_append(ptr, num)      cmd(CMD_APPEND);cmd(ptr);cmd(num)
259
260 /**
261  * decompress data into memory
262  */
263 #define cmd_inflate(ptr) cmd(CMD_INFLATE);cmd(ptr)
264
265 /**
266  * set the current matrix to identity
267  */
268 #define cmd_loadidentity() cmd(CMD_LOADIDENTITY)
269
270 /**

```



```

271     * write the current matrix as a bitmap transform
272     */
273 #define cmd_setmatrix() cmd(CMD_SETMATRIX)
274
275     /**
276     * retrieves the current matrix coefficients
277     */
278 #define cmd_getmatrix(a, b, c, d, e, f) cmd(CMD_GETMATRIX); unsigned short cmd_getmatrix_result_ptr=cmd_offset;cmd(0);cmd(0);cmd(0);cmd(0);cmd(0);cmd(0);cmd(0);cmd_exec();cmd_wait();a=rd32(cmd_getmatrix_result_ptr);b=rd32((cmd_getmatrix_result_ptr+4)%4096);c=rd32((cmd_getmatrix_result_ptr+8)%4096);d=rd32((cmd_getmatrix_result_ptr+12)%4096);e=rd32((cmd_getmatrix_result_ptr+16)%4096);f=rd32((cmd_getmatrix_result_ptr+20)%4096)
279
280     /**
281     * Get the end memory address of inflated data
282     */
283 #define cmd_getptr(result) cmd(CMD_GETPTR); unsigned short cmd_getptr_result_ptr=cmd_offset;cmd(0);cmd_wait();result=rd32(cmd_getptr_result_ptr)
284
285     /**
286     * apply a scale to the current matrix
287     */
288 #define cmd_scale(sx, sy) cmd(CMD_SCALE);cmd(sx);cmd(sy)
289
290     /**
291     * apply a rotation to the current matrix
292     */
293 #define cmd_rotate(a) cmd(CMD_ROTATE);cmd(a)
294
295     /**
296     * apply a translation to the current matrix
297     */
298 #define cmd_translate(tx, ty) cmd(CMD_TRANSLATE);cmd(tx);cmd(ty)
299
300     /**
301     * load a JPEG image
302     */
303 #define cmd_loadimage(ptr, options) cmd(CMD_LOADIMAGE);cmd(ptr);cmd(options)
304
305     /**
306     * set co-processor engine state to default values
307     */
308 #define cmd_coldstart() cmd(CMD_COLDSTART)
309
310     /**
311     * trigger interrupt INT_CMDFLAG
312     */
313 #define cmd_interrupt(ms) cmd(CMD_INTERRUPT);cmd(ms)
314

```



```

315     /**
316     * read a register value
317     */
318     #define cmd_regread(ptr, result)    cmd(CMD_REGREAD);cmd(ptr);cmd      ↗
        (result)
319
320     /**
321     * execute the touch screen calibration routine
322     */
323     #define cmd_calibrate() cmd(CMD_CALIBRATE);cmd(command: 0) //;unsigned  ↗
        short cmd_calibrate_result_ptr=cmd_offset;cmd(0)
324     #define cmd_calibrate_result() rd32(cmd_calibrate_result_ptr)
325
326     /**
327     * start an animated spinner
328     */
329     #define cmd_spinner(x, y, style, scale) cmd(CMD_SPINNER);wr16(RAM_CMD  ↗
        +cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,
        y);cmd_increment2();wr16(RAM_CMD+cmd_offset, style);cmd_increment2  ↗
        ();wr16(RAM_CMD+cmd_offset, scale);cmd_increment2()
330
331     /**
332     * start an animated screensaver
333     */
334     #define cmd_screensaver() cmd(CMD_SCREENSAVER)
335
336     /**
337     * start a continuous sketch update
338     */
339     #define cmd_sketch(x, y, w, h, ptr, format) cmd(CMD_SKETCH);wr16      ↗
        (RAM_CMD+cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,
        y);cmd_increment2();wr16(RAM_CMD+cmd_offset, w);cmd_increment2();wr16  ↗
        (RAM_CMD+cmd_offset, h);cmd_increment2();cmd(ptr);wr16(RAM_CMD
        +cmd_offset, format);cmd_increment2()  ↗
340
341     /**
342     * stop any spinner, screensaver or sketch
343     */
344     #define cmd_stop() cmd(CMD_STOP)
345
346     /**
347     * set up a custom font
348     */
349     #define cmd_setfont(font, ptr) cmd(CMD_SETFONT);cmd(font);cmd(ptr)
350
351     /**
352     * track touches for a graphics object
353     */
354     #define cmd_track(x, y, w, h, tag) cmd(CMD_TRACK);wr16(RAM_CMD      ↗
        +cmd_offset, x);cmd_increment2();wr16(RAM_CMD+cmd_offset,
        y);cmd_increment2();wr16(RAM_CMD+cmd_offset, w);cmd_increment2();wr16  ↗
        (RAM_CMD+cmd_offset, h);cmd_increment2();wr16(RAM_CMD+cmd_offset,
        tag);cmd_increment2()  ↗

```

```
355
356     /**
357     * take a snapshot of the current screen
358     */
359 #define cmd_snapshot(ptr) cmd(CMD_SNAPSHOT);cmd(ptr)
360
361     /**
362     * play device logo animation
363     */
364 #define cmd_logo() wr32(RAM_CMD + 0, CMD_LOGO) //cmd(CMD_LOGO)
365
366
367 #ifdef __cplusplus
368 }
369 #endif
370
371 #endif /* MC32GESTSPIFT800_H */
372
373
```